CCP6124 Object-oriented Programming and Data Structures
Trimester 2610

Project

Virtual Machine and Assembly Language Interpreter

1. Introduction

In this assignment, you will implement an assembly language interpreter that runs assembly language instructions according to a simplified virtual machine architecture. The interpreter must be designed using **Object-Oriented Programming (OOP)** principles and must explicitly utilise standard **Data Structures (DS)**. The interpreter is expected to execute an assembly program and generate a display of the content of the virtual machine after execution.

The assembly language for the given virtual machine supports various operations such as:

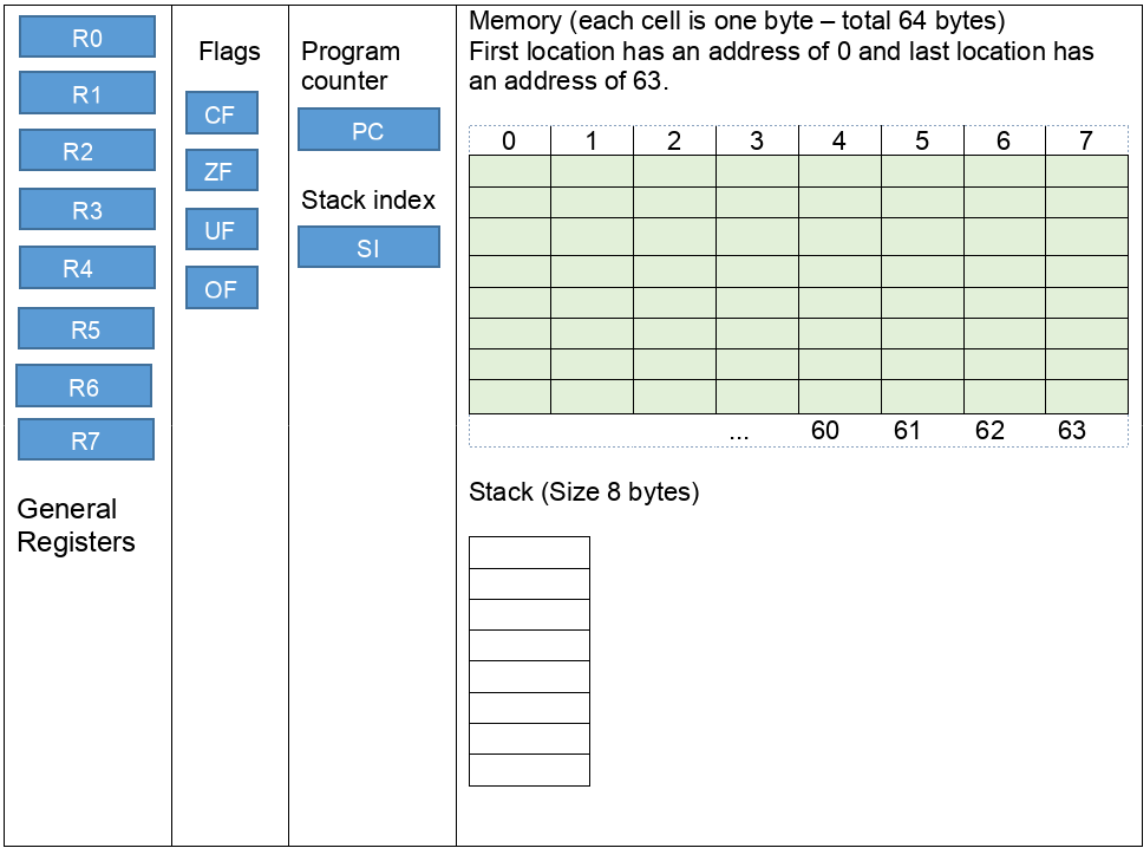
1. Move operations,
2. Mathematical operations,
3. Rotation and shifting operations,
4. Basic I/O operations,
5. Loading data from memory,
6. Storing data to memory.
7. Stack operations

2. Virtual Machine Architecture:

The virtual machine includes 8 data general registers and a program counter where each of them is one byte wide (char), 4 flag bits (Overflow, Underflow, Carry, and Zero), and a main memory of 64 bytes addressed from 0 to 63.

Your virtual machine architecture will include the following components:

- **Data Registers:** 8 general-purpose data registers (R0–R7), each 1 byte (signed char, -128 to 127).
- **Program Counter (PC):** 1 byte, starts at 0, increments after each instruction.
- Stack Index (SI) Register: 1 byte, starts at 0, incremented when items are pushed to the stack and decremented when an item is popped from the stack.
- **Flags:** 4 flag bits (Overflow, Underflow, Carry, Zero).
- **Memory:** 1-dimensional array of 64 signed bytes (addresses 0–63).
- **Runner (Interpreter):** Reads `.asm` files, decodes instructions, and executes them via the CPU class.



2.1. Data Registers:

Eight general-purpose data registers: R0, R1, R2, R3, R4, R5, R6, and R7. Each register is represented as one byte (8 bits) in size. The contents of these registers are updated after executing the assembly instruction by the runner (program that executes the instructions and update virtual machine components). These registers can contain **signed bytes** (values between -128 and 127) only. Negative numbers are represented using 2's complement.

2.2. Program Counter (PC)

The program counter starts always with the value 0 (pointing to the first assembly instruction in the program), then it is incremented whenever the runner (interpreter) executes an assembly instruction (loaded from a text file with the extension ".asm"). If the program has 10 instructions, it will start with value 0 representing the execution of the first instruction, and 10 after executing the last instruction.

2.3. Flags:

- Overflow Flag (OF): A single bit flag (or a byte – your choice) indicating arithmetic overflow. It is set when an arithmetic operation results in a value greater than 127.
- Underflow Flag (UF): A single bit flag (or byte) indicating arithmetic underflow. Set when an arithmetic operation results in a value smaller than -128.
- Carry Flag (CF): A single bit flag (or a byte) indicating carry in arithmetic operations. Set to 1 if the calculated result of the instructions (ADD, SUB, MUL, DIV, INC, DEC) exceeds the capacity of 8 bits.
- Zero Flag (ZF): A single bit flag (or a bite) indicating the result of an operation is zero. Set when the result of an operation is zero.

2.4. Memory:

The memory can be represented as a one-dimensional array of 64 signed bytes. Memory addresses are numbered from 0 to 63. Memory can be accessed only by a **load** or **store** assembly operations.

3. Assembly Language Runner (interpreter):

The interpreter reads an assembly program line by line from a ".asm" file (text file) from disk. The runner reads the file into a dynamic array/vector of strings, where each array element stores a line form the file. Each line contains only one single instruction. If more than one instruction found on one line, the interpreter generates an error message and exits. Empty lines must be ignored. The runner then executes the instructions from the array/vector of instructions one by one.
The PC is initialised to 0 when loading a new program and is updated after the execution of any statement. At the end of execution, it will produce a dump (display the content of the virtual machine) showing of all registers and memory to the screen in addition to the output window (results of the I/O operations).

3.1. The assembly language instructions:

The assembly language is expected to support I/O operations, arithmetic operations, rotation, shifting, loading from memory (direct and register-indirect addressing), and storing into memory.

3.2. Input operations:

INPUT <Destination Register>

Reads a value from the keyboard and store it into destination register

INPUT R0
The instruction INPUT causes the terminal to display a **"?"** mark at the beginning of a new line waiting for an input. The interpreter should manage input errors so that inputs are in the range (-128..127). The interpreter will store the value to register R0 and check for overflow, underflow, and zero and update the corresponding registers. • If the input has value that is greater than 127, it will set the OF flag to true. • If the input has a value that is smaller than -128, it will set the UF flag to true. • If the input has an ASCII code that is equal to 0, it will set the ZF flag to true.

3.3. Output operations:

DISPLAY <Source Register>

Writes to the screen the value inside the register source register.

DISPLAY R0
This instruction will print out to the terminal the value found in register R0.

3.4. MOV operations:

MOV <Destination Register> , <Source Register>

Copies the values stored in the source register to the destination register. The **mov** operation supports three modes only; these modes are explained in the following 3 examples.

The following shows the three types of allowed **mov** operations.

MOV R0, 10	**MOV R0, R1**	**MOV R3, [R1]**
Stores the value 10 to register R0	Copies the value stored in register R1 to R0	Copies the value with the address stored in R1 to register R3. R1 is contains the address of a byte in memory. The square brackets [] indicates that the value in R1 is an address, and the move operation reads the memory at that address, fetch the value from memory and store it in R3.

3.5. Arithmetic Operations (Arithmetic operations to be implemented in decimal – no need for binary representations), if implemented as binary registers with 1s and 0s, it is considered as bonus:

- **The ADD instruction**

  ADD <DestinationRegister> , <SourceRegister>

  Adds the value of source register to destination register and store it in destination.

- **The SUB instruction**

  SUB <DestinationRegister> , <SourceRegister>

  Subtract the value of source register from destination register and store it in destination.

- **The MUL instruction**

  MUL <DestinationRegister> , <SourceRegister>

  Multiply the value of source register by the value of destination register and store it in destination.

- **The DIV instruction**

  DIV <DestinationRegister>, <SourceRegister>

  Divides the value of source register by the value of destination register and store it in destination.

The above 4 instructions operate in a similar manner. These operations only operate on the 8 data registers directly. Both parameters for these instructions are registers.

ADD R2, R0

- The instructions add the value found in the register R0 to the value found in R2 and stores the result in R2.
- The runner will also update the flags based on the results computed in R2.
  - If the value computed in R2 is less than -128 UF is set (true to UF).
  - If the value exceeds 127, the OF is set to true.
  - If the value of R2 is 0, then the ZF is set to true.

(MUL R0, R2, DIV R1, R3, SUB R0, R1)

3.6. Increment and Decrement operations:

- **The INC instruction**

INC <Destination Register>

Adds 1 to the value stored in the destination.

- **The DEC instruction**

DEC <DestinationRegister>

Subtracts 1 from the value stored in the destination.

INC R0	Adds 1 to the register R0
DEC R7	Subtracts 1 from the register R7

In both cases it checks for overflow (OF), underflow (UF), and zero (ZF) contents and sets the value of the flags in a similar way to what was shown earlier in the math operations.

3.7. Rotate Operations (Your own algorithm):

The runner will convert the value of a register into binary signed number, then perform the rotation and shift operations, after that it is converted back to a decimal number and stored back to the register.

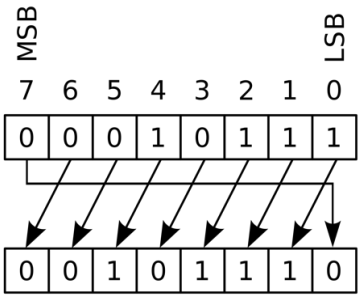
- **The ROL instruction**

ROL <DestinationRegister> , <count>

Rotate the bits in destination left by 'count' positions.

ROL R1, 1

The value in R1 is converted to a binary value first, then the Bits are rearranged in a similar manner shown in the example below. After the rotation is done, convert back to decimal and store into R1.



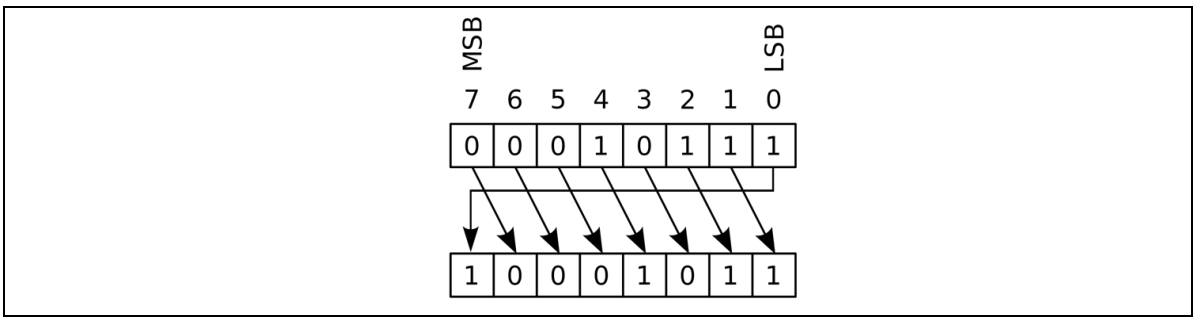
- **The ROR instruction**

ROR <DestinationRegister> , <count>

Rotate the bits in destination right by 'count' positions.

ROR R1, 1

The value in R1 is converted to a binary value first, then the Bits are rearranged in a similar manner shown in the example below. After the rotation is done, convert back to decimal and store into R1.



3.8. Shift Operations (Your own algorithm):

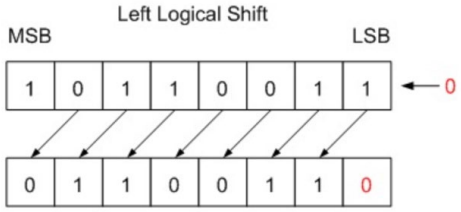
- **The SHL instruction**

SHL <DestinationRegister> , <count>

Shifts the bits in destination left by 'count' positions, filling shifted bits positions with zeros. Shifting the number 7 times will result in 0 stored in the destination register.

SHL R1, 1

The value in R1 is converted to a binary value first, then the Bits are rearranged in a similar manner shown in the figure below. After the shifting is done, convert back to decimal and store into R1.



- **The SHR instruction**

SHR <DestinationRegister> , <count>

Shift the bits in **<DestinationRegister>** right by **<count>** positions, filling with zeros.

SHR R1, 1

The value in R1 is converted to a binary value first, then the Bits are rearranged in a similar manner shown in the figure below. After the shifting is done, convert back to decimal and store into R1.

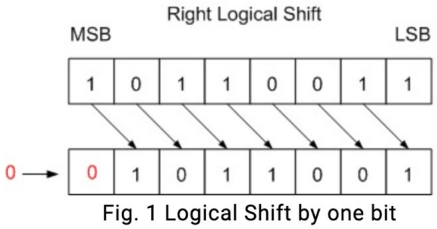


Fig. 1 Logical Shift by one bit

3.9. Load and Store Operations:

The main purpose of these operations is to move data between data registers and memory and vice versa.

- **The LOAD instruction** (1)

LOAD <DestinationRegister> , <address>

Load the value at memory address **<address>** to destination register.

LOAD R1, [20]
Read the memory location 20 and copy the value to the register R1

- **The LOAD instruction (2)**

LOAD <DestinationRegister>, <[SourceRegister]>

Load the value stored in the memory address found in the source register and store it in the destination register.

LOAD R1, [R2]
Read the memory location with the address stored in R2 and copy the value found in the memory location into the register R1. R2 contains a memory address.

- **The STORE instruction (1)**

STORE <Destination Register>, <memoryAddress>

Store the value in found in the source register into the memory address.

STORE R1, 43
Store the value found in register R1 to the memory location 43.

- **The STORE instruction (2)**

STORE <[DestinationRegister]> , <**SourceRegister**>

Store the value in source register to the memory address stored in the destination register.

STORE [R2], R1
Store the value found in R1 to the memory location with address found in R2. R2 contains a memory address.

3.10. The Flag Reset instruction

RESET <Target Flag>

Resets the flag to 0

RESET CF
Stores 0 to the Carry Flag (CF) (RESET UF RESET OF RESET ZF)

- The flags should be updated after each operation for all types of operation that changes the value of a destination register.

3.11. Stack Operations

- **Push**

PUSH <REGISTER>

PUSH R0
Stores the value stored in R0 to the stack (system stack) and increment the SI register

- **Pop**

POP <REGISTER>

POP R0
Remove the top of the stack and store it into R0 and decrement the SI register

Any attempt to pop from an empty stack will cause system to stop and crash.

4. OOP Class Design Requirements

The project must be designed as an object hierarchy with clear relationships. You are required to implement the following class structure (or equivalent):

Class	Purpose	Relation
Register	Encapsulates an 8-bit signed value and flag update logic	Base class
GeneralRegister	Represents R0–R7 registers	Derived from 'Register'
FlagRegister	Manages individual flag bits (CF, OF, UF, ZF)	Aggregated by 'CPU'
Memory	Handles storage and addressing logic over a vector of bytes	Composition inside 'CPU'
Instruction	Abstract base class for all assembly commands	Parent class for 'ADD', 'MOV', etc.
ArithmeticInstruction, IOInstruction, ShiftInstruction	Derived instruction classes implementing specific behaviors	Polymorphic use in 'Runner'
CPU	Contains registers, memory, PC, executes instructions	Composed of other classes
Runner	Loads programs, decodes instructions, delegates execution to `CPU`	Coordinates the system

Explicit OOP Requirements:

- **Encapsulation:** All class members must be private; access via getters/setters.
- **Inheritance:** Use base abstract classes like Instruction with virtual execute() methods.
- **Polymorphism:** The interpreter must store instructions as **vector<Instruction*>** (or similar) and execute dynamically via virtual polymorphism.
- **Composition/Aggregation:** Illustrate how CPU owns (composition) Memory but uses (aggregation) other components.

5. Data Structures Requirements

You must explicitly use standard data structures logically within the problem logic. The following structures are required:

- **Vector/List:** For instruction storage and iteration (loading the .asm file into a dynamic array).
- **Stack:** Must be used within the program logic (e.g., for tracking execution history, managing nested operations, or internal temporary storage).
- **Queue:** can be used to store the program.

Students must implement these data structures and use their own classes and not use the ones from the STL.

6. Program Input and Output:

The program is expected to read an assembly program, execute it, and store the results to a file and display to the screen.

Sample Program:

```
MOV    R1, 5
ADD    R1, 6
MOV    R3, R1
MUL    R3, 4
STORE  20, R3
```

Step by step executing the program above

Start program

R0	R1	R2	R3	R4	R5	R6	R7	PC
								0

MOV R1, 5

R0	R1	R2	R3	R4	R5	R6	R7	PC
	5							1

ADD R1, 6

R0	R1	R2	R3	R4	R5	R6	R7	PC
	11							2

MOV R3, R1

R0	R1	R2	R3	R4	R5	R6	R7	PC
	11		11					3

MUL R3, 4

R0	R1	R2	R3	R4	R5	R6	R7	PC
	11		44					4

STORE 20, R3

R0	R1	R2	R3	R4	R5	R6	R7	PC
	11		44					5

End of program

Memory (0 .. 63)

0	1	2	3	4	5	6	7
				44			
..	..	..	..	60	61	62	63

STORE 20, R3

R0	R1	R2	R3	R4	R5	R6	R7	PC
	11		44					6

The results should be like the sample below:

It must be identical in format to the output below (matches letter casing, no extra spaces, only # is used
Sample output file:

#Begin#
#Registers#**0000#0011#0000#0044#0000#0000#0000#0000#**
#Flags#**0#0#0#0#**
#PC#000**6#**
#Memory#
#0000#0000#0000#0000#0000#0000#0000#0000#
#0000#0000#0000#0000#0000#0000#0000#0000#
#0000#0000#0000#0000#**0044#**0000#0000#0000#
#0000#0000#0000#0000#0000#0000#0000#0000#
#0000#0000#0000#0000#0000#0000#0000#0000#
#0000#0000#0000#0000#0000#0000#0000#0000#
#0000#0000#0000#0000#0000#0000#0000#0000#
#0000#0000#0000#0000#0000#0000#0000#0000#
#End#

Conclusion:

Through this comprehensive assignment, you will gain practical experience in using objects, inheritance, composition, aggregation, and polymorphism, and in writing your own data structures as you create a runner program to execute assembly programs and display the virtual machine's state.

7. Assignment Deliverables:

7.1. Report

Architecture Report:

Describe the general architecture of the virtual machine, the runner, and any classes required for the solution. Must show composition, inheritance, and aggregation relationships between classes.

Algorithms used:

Detailed explanations of the main algorithms used in the runner (Interpreter). All algorithms must be presented using UML activity diagrams with a general explanation for each one of them. An overall UML class diagram that represents the components of the solution where callers and called modules need to be specified and clearly explained.

Assembly Language Syntax and Examples:

You must provide at least 3 full assembly programs (compute some real problems such as summing 5 values, finding the average of 4 values, calculate the factorial of 4, etc.) with each demonstrating all the commands to be implemented and sample screen shots of the resulted output of the interpreter. Explain what each program is expected to do.

The examiners will be testing your runners with their own assembly code and it must produce identical results to our implementation of the assignment following the assignment specifications.

Flags Handling Explanation:

Detailed explanation of how flags are updated for each operation in addition to sample screen shots to demo your understanding and full functionality. Explain by showing sample code).

The Runner Demo:

The report must show at least one sample run of an example assembly program, step by step execution, showing the results by showing the memory and registers after each command execution. This must be shown with screenshots after each command.

A user manual on how to compile and run your code via the command line. (So that lecturers marking it will not need to have the same IDE as you.) If the program cannot be compiled and run from the command line, 25 marks for source code will be lost.

7.2 Comments in the source code

The source code must be properly commented, including information of who wrote each class or template in the program. If more than one student wrote a particular class or template, then each part should be commented to clearly identify who wrote that part. **You will lose marks if you do not do this**.

7.3 Video

A video recording that will include all the members explaining their own (programming) contribution and to demo compiling and running the program in addition the assembly instructions and part of the code implemented by them. Each student must state clearly his contributions in the assignment. Students who are not contributing to the source code will get 0 for the assignment.

Assignment Submission

The assignment contains the following components:

1. The assignment must be submitted in group of minimum two persons to four persons.

2. The due date for the assignment submission is **4 July 2026, 12.00AM (Friday midnight or Saturday morning)**.

3. The source code must be stored in a single C++ file and must be named with the <tutorial section number> followed by an underscore then <group number> followed by ".cpp" (i.e. **TT01_G01.cpp**). No multiple files are allowed to be submitted.

4. The **report** (must be **PDF** format only). Any other format will cause you to get 0 for the report. The report (named also the same as the source code name **TT01_G01.pdf**).

5. **Video recording** (must be **mpeg** format i. e. **TT01_G01.mpeg**) and any other format will cause you to get 0 on the video component.

6. The report and source code must placed in a folder with exactly the same name as the file (i.e. **TT01_G01**).

7. A zipped file of the folder and the files inside it must be submitted. Only ".zip" (**TT01_G01.zip**) files are allowed. Any other format will cause you to get a 0 for the assignment.

Do not email us your project and make sure not to leave your submission until last minute.

Late submission policy:

- **20%** of the total assignment mark will be deducted if the project is submitted within the first 12 hours after submission deadline.

- **30%** will be deducted if the submission is within (12:00 - 23:59) hours late.
- **50%** will be deducted if it is submitted **2 days late**.
- **0%** will be awarded if passed these deadlines.

If you have submitted a version, but then change it, you can re-submit until the cut-off date for each of these items, and the new version will replace the old version.

Any re-submission after the deadline will be treated as new submission and penalties are applied.

CCP6214 Assignment Evaluation Form (40%)

Group ID		
ID	Name	Individual Grades

Assignment implementation (15%)

Item	**Max Marks**	**Actual Marks**
Code quality and style (Inline comments, function and class comments, indentation, following proper C++ naming and styling conventions) ***Any violation is penalised by reduction of -0.5 mark.***	2	
Proper class declarations and encapsulation that maps to the architecture in the report. (***Any violation is penalised by reduction of -0.5 mark).***	2	
Must use properly implementation (Lists, Queues, Stack) and make use of them in your assignment. (***Any missing data structure is penalised by -1 mark).***	3	
Inheritance Hierarchy must be used in the code in at least two different places in the assignment. (***Any missing feature is penalised by deducting -1 mark).***	3	
Polymorphism must be used in the code in at least two different places in the assignment. (***Any missing feature is penalised by deducting -1 mark).***	3	
Provided 3 example ideas of the program. (***any missing program is penalised by -0.33 mark)***	1	
Functions must not exceed 35 lines of code. ***(Any function found to exceed 35 lines a penalty of -0.5 will be imposed)***	1	

Program Run and meeting the requirements (10%)

Item	**Maximum marks**	**Actual Marks**
Instruction are all implemented correctly and produce the right results ***(Any failed instruction is penalised with -0.5 mark).***	8	
Flags are implemented correctly and produce the expected output. ***(Each flag worth 0.5 mark).***	2	

Report (10% – must submit otherwise fail the assignment)

Item	**Max Marks**	**Actual Marks**
Overall class diagram and architecture - 0.5 for any missing class	5	
All class relationships are correct - 0.5 for each wrong relationship	4	
User manual (how to compile and run the code)	1	

Interview (5%)

Item	**Maximum marks**	**Actual Marks**
Student are asked 3 questions each about the assignment implementation (individual marks)	5	